SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS: **Total Marks: 25**

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

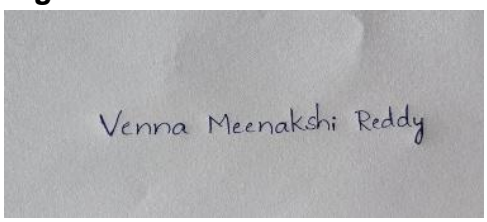
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

4.An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5.A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I **VENNA MEENAKSHI REDDY(192471048)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

1. A company needs to develop a web service for a mobile and web application. Analyze both REST and SOAP approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

A company developing a mobile and web application can use either REST (Representational State Transfer) or SOAP (Simple Object Access Protocol) to provide web services.

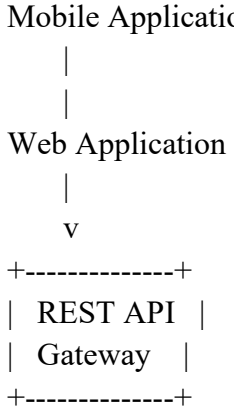
REST vs SOAP Comparison

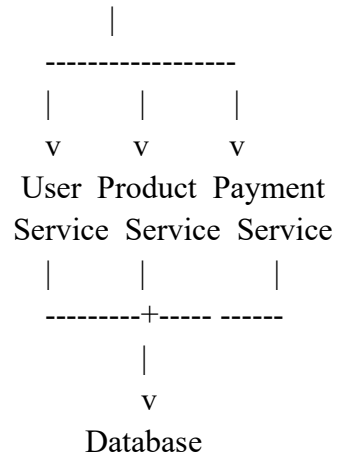
Feature	**REST**	**SOAP**
Architecture	Resource-based	Protocol-based
Data formats	Mainly JSON, also XML	Mainly XML
Performance	Faster and lightweight	Slower due to XML processing
Scalability	Highly scalable	Comparatively complex to scale
Security	HTTPS, OAuth, JWT, API keys	WS-Security, HTTPS
Ease of development	Simple	More complex
Mobile applications	Very suitable	Less suitable
Bandwidth usage	Low	Higher
Caching	Easily supported	More difficult
Reliability	Depends on implementation	Supports advanced enterprise features

Proposed Solution: REST API

For a modern mobile and web application, REST is the most appropriate choice.

Architecture





Justification

1.Performance:

REST commonly uses JSON, which is smaller and easier to process than SOAP XML messages. This reduces bandwidth consumption and improves response time, especially for mobile applications.

2.Scalability:

REST services are generally stateless. Each request contains the information required to process it, allowing requests to be distributed among multiple servers using a load balancer.

3.Security:

REST can use HTTPS/TLS to encrypt communication. Authentication can be implemented using OAuth 2.0, JWT tokens, or API keys. Authorization can be implemented using role-based access control.

4. Flexibility:

REST supports different clients such as Android, iOS, browsers, and other applications.

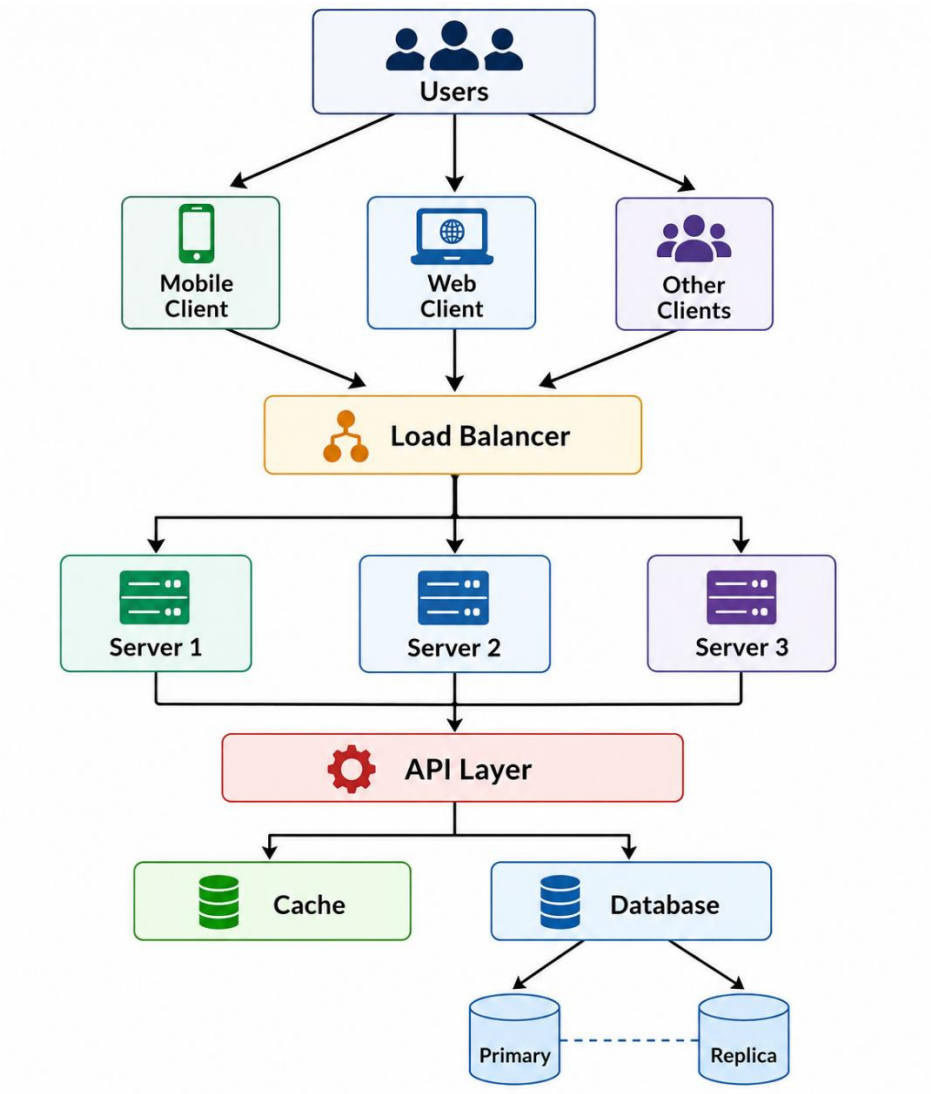
Conclusion

REST should be selected because the application requires good performance, scalability, lightweight communication, and support for mobile and web clients. SOAP would be more suitable for systems requiring strict enterprise standards and advanced WS-* features.

2. An online platform expects a large number of users accessing its services simultaneously. Design a scalable service architecture showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

An online platform with a large number of simultaneous users requires an architecture that can handle high traffic without reducing performance.

Basic Scalable Architecture

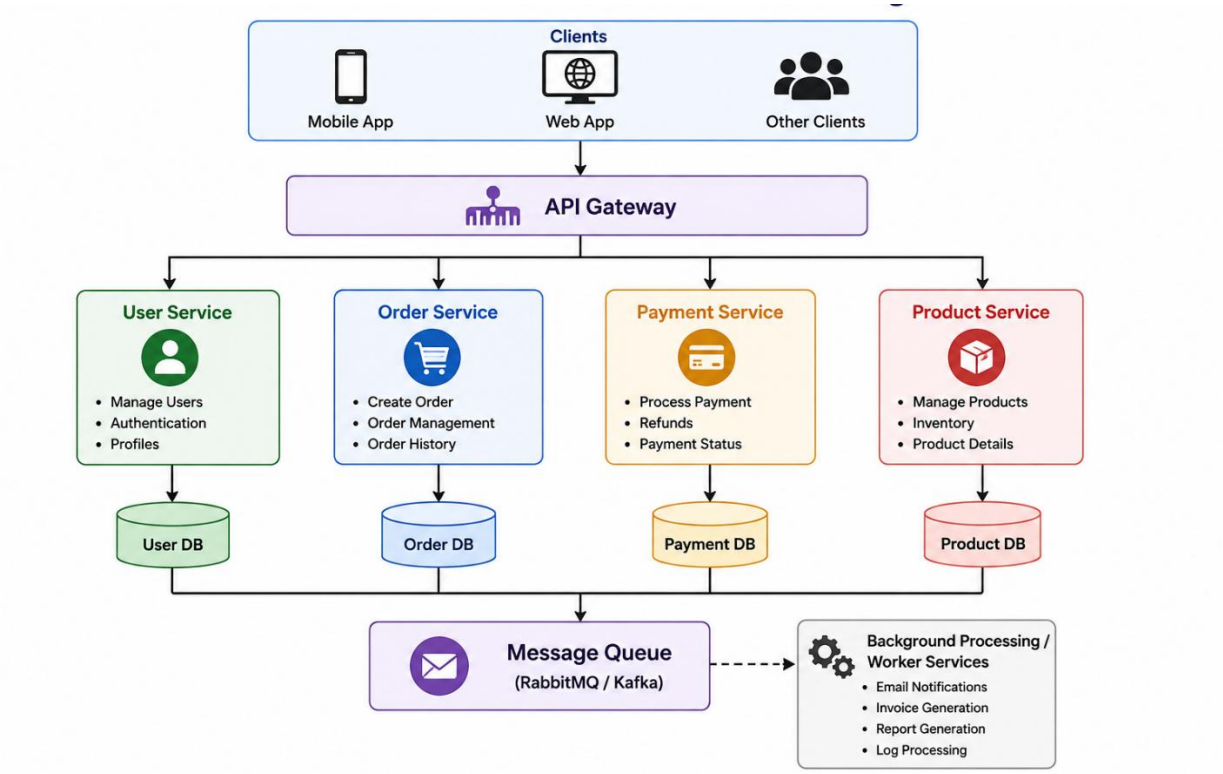


Interaction

1. The client sends a request.
2. The load balancer receives the request.
3. The load balancer distributes requests among available servers.
4. The application servers process business logic.
5. The API layer communicates with required services.
6. Frequently accessed information can be retrieved from a cache.
7. Persistent information is stored in the database.

Alternative Design: Microservices Architecture

A better solution for very large platforms is a microservices architecture.



Advantages

- Individual services can be scaled independently.
- Failure of one service does not necessarily bring down the entire system.
- Different services can use different technologies.
- Load can be distributed across multiple servers.
- Caching reduces database load.
- Database replicas can handle large numbers of read requests.

Conclusion

For a small or medium system, a load-balanced multi-server architecture is sufficient. For a very large platform, microservices + API Gateway + caching + database replication + message queues provides better scalability and fault isolation.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and select the most suitable protocol. Justify your selection based on security, reliability, and performance.

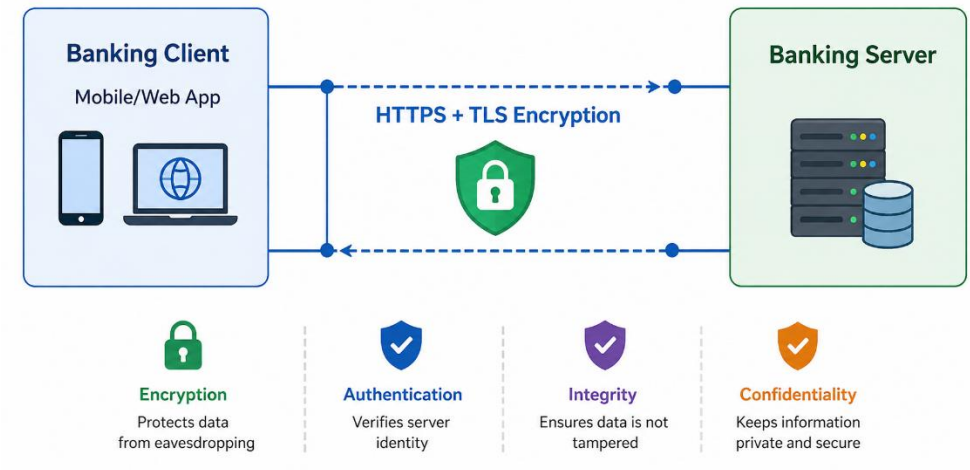
A banking application handles highly sensitive information such as account numbers, passwords, transaction details, and financial data. Therefore, communication between the client and server must provide confidentiality, integrity, authentication, and reliability.

Available Protocols

Protocol	**Security**	**Reliability**	**Suitability**
HTTP	Low	Good	Not suitable for banking
HTTPS	High	Good	Highly suitable
FTP	Low/Moderate	Good	Not suitable
TCP	No encryption by itself	Very high	Requires security layer
UDP	No encryption by itself	Lower	Not suitable for financial transactions
TLS	Very high	High	Used with HTTPS

Selected Protocol: HTTPS with TLS

The most suitable solution is HTTPS using TLS (Transport Layer Security).



Why HTTPS/TLS?

1.Confidentiality:

TLS encrypts information during transmission. Attackers cannot easily read passwords or transaction information if the traffic is intercepted.

2. Authentication:

The server uses a digital certificate to prove its identity to the client. This helps prevent man-in-the-middle attacks.

3. Data Integrity:
TLS provides mechanisms to detect whether transmitted data has been modified.
4. Reliability:
HTTPS normally operates over TCP, which provides reliable and ordered delivery of data.
5.Performance:
Modern TLS versions, particularly TLS 1.3, reduce connection overhead and provide strong security with good performance.
Additional Banking Security
HTTPS alone should not be considered sufficient. The application should also use:

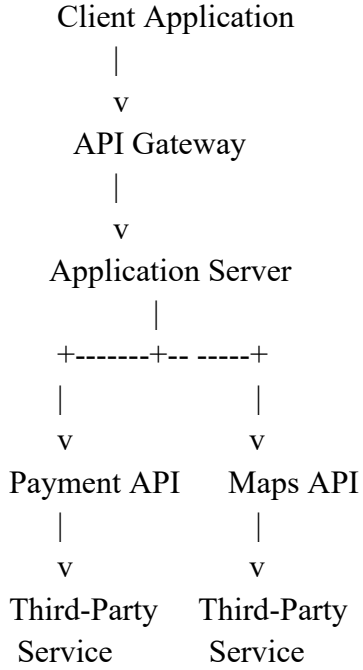
- Multi-factor authentication
- Strong password policies
- JWT/session-based authentication
- Role-based authorization
- Transaction signing or OTP where appropriate
- Rate limiting
- Secure session management
- Server-side validation
- Audit logging

Conclusion
HTTPS with TLS is the most suitable communication protocol for banking applications because it provides encryption, authentication, integrity, and reliable communication while maintaining good performance.

4.An application needs to integrate with third-party services such as payment gateways or maps. Design an API integration strategy, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.

Suppose an application needs to integrate with a payment gateway or map service. The application should use a well-defined API integration strategy.

Proposed API Integration Architecture



Request Handling

The application can follow these steps:

Step 1 – Client Request

The client sends a request to the application server.

Example:

POST /payment

Step 2 – Validation

The server validates:

- Required fields
- User identity
- Amount
- Transaction details
- Request format

Step 3 – Authentication

The application authenticates with the third-party service using an appropriate mechanism such as:

- API key
- OAuth 2.0
- Access token
- Signed request

Sensitive credentials should be stored securely and should not be exposed to the client.

Step 4 – API Request

The server sends a request to the external service over HTTPS.

Application Server
|
| HTTPS + Authentication
v
Third-Party API

Step 5 – Response Processing

The application receives the response and checks:

- HTTP status code
- Response body
- Transaction/reference ID
- Error code
- Authentication status

Step 6 – Client Response

The server sends a simplified and safe response to the client.

Third-Party API
|
v
Application Server
|
v
Client Application

Alternative Approach: Asynchronous Integration

For operations that do not need an immediate response, a **message queue** can improve reliability.

Client
|
v
Application
|
v
Message Queue
|
v
Integration Worker
|
v
Third-Party API

The worker can retry failed requests without forcing the user to wait.

Benefits

- Protects third-party credentials
- Reduces dependency between client and external services
- Allows request validation
- Supports retries
- Provides centralized logging
- Makes it easier to replace third-party providers
- Improves reliability during temporary third-party failures

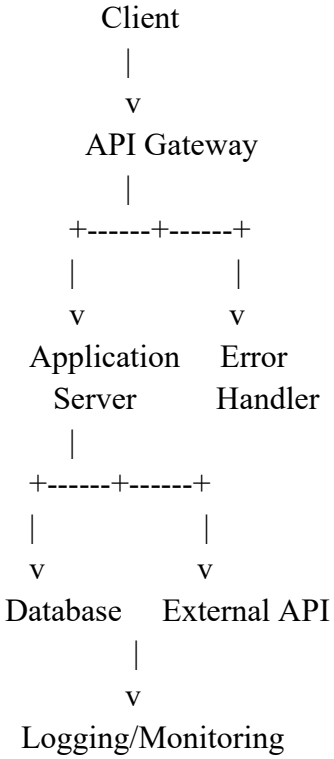
Conclusion

A server-side API integration through an API gateway/service layer is recommended. For high-volume or non-real-time operations, asynchronous processing with a message queue provides better efficiency and reliability.

5.A web service frequently encounters errors during client requests. Design an error handling and fault tolerance mechanism. Propose alternative strategies to improve system reliability and ensure smooth user experience.

A web service may experience errors because of invalid client requests, network failures, database failures, overloaded servers, or third-party service failures. Therefore, an effective error-handling mechanism is required.

Proposed Architecture



Error Handling Mechanism

1. In Validation

2. The server should validate all client requests before processing them.

For example:

Invalid input

|

v

HTTP 400 Bad Request

This prevents incorrect data from reaching the business logic or database.

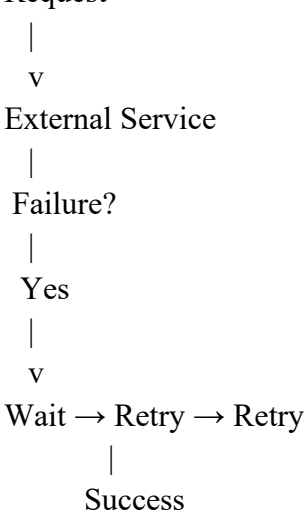
2. HTTP Status Codes

Appropriate status codes should be returned.

Error	**Status Code**
Invalid request	400
Authentication required	401
Access denied	403
Resource not found	404
Too many requests	429
Server error	500
Service unavailable	503
Gateway timeout	504

3. Retry Mechanism

Temporary failures can be handled using controlled retries.



Exponential backoff should be used so that repeated failures do not overload the service.

4. Timeout

Every external request should have a timeout. The application should not wait indefinitely for an unavailable server.

5. Circuit Breaker

A circuit breaker can stop repeated requests to a failing service.

Normal
|
v
Closed
|
Repeated failures
|
v
Open
|
Wait
|
v
Half-Open
|
Success ---> Closed
Failure ---> Open

This prevents cascading failures.

6. Logging and Monitoring

The system should maintain logs containing:

- Error type
- Timestamp
- Request ID
- Service involved
- Status code
- Failure reason

Monitoring tools can generate alerts when the error rate becomes high.

Alternative Strategies

1. Redundant servers:

Multiple application servers can provide service if one server fails.

2. Load balancing:

Requests can be redirected from failed or overloaded servers to healthy servers.

3. Database replication:

A replica can provide data if the primary database becomes unavailable.

4. Message queues:

Failed background operations can be stored and retried later.

5. Graceful degradation:

If a non-critical service fails, the application can continue providing essential features.

For example, if a recommendation service is unavailable, the shopping application can still allow users to browse and purchase products.

Conclusion

A combination of input validation, proper HTTP status codes, timeouts, retries with exponential backoff, circuit breakers, logging, monitoring, load balancing, and graceful degradation provides strong fault tolerance and a smooth user experience.